Experiment 5

5. Construct a scheduling program with C that selects the waiting processwith the highest priority to execute next.

```
#include <stdio.h>
struct process {
   char process_name;
   int burst_time;
   int priority;
   int waiting_time;
   int turnaround_time;
};
int main() {
   int n, total = 0;
   int position;
   float average_waiting_time, average_turnaround_time;
   struct process temp;
   printf("Enter the total number of processes: ");
   scanf("%d", &n);
   struct process p[n];
   for (int i = 0; i < n; i++) {
      p[i].process_name = 'A' + i;
      printf("\nEnter the burst time of process %c: ", p[i].process_name);
      scanf("%d", &p[i].burst_time);
      printf("Enter the priority of process %c: ", p[i].process_name);
      scanf("%d", &p[i].priority);
   }
   for (int i = 0; i < n; i++) {
      position = i;
      for (int j = i + 1; j < n; j++) {
         if (p[j].priority < p[position].priority) {
            position = j;
```

```
      }
    }

    temp = p[i];
    p[i] = p[position];
    p[position] = temp;
}
p[0].waiting_time = 0;

for (int i = 1; i < n; i++) {
    p[i].waiting_time = p[i - 1].waiting_time
                + p[i - 1].burst_time;
}
total = 0;
int total_turnaround_time = 0;

printf("\nProcess\tBurst Time\tPriority\tWaiting Time\tTurnaround Time\n");

for (int i = 0; i < n; i++) {
    p[i].turnaround_time = p[i].burst_time + p[i].waiting_time;

    printf("%c\t%d\t\t%d\t\t%d\t\t%d\n",
          p[i].process_name,
          p[i].burst_time,
          p[i].priority,
          p[i].waiting_time,
          p[i].turnaround_time);

    total += p[i].waiting_time;
    total_turnaround_time += p[i].turnaround_time;
}
```

```
    average_waiting_time = (float) total / n;
    average_turnaround_time = (float) total_turnaround_time / n;

    printf("\nAverage Waiting Time = %.2f", average_waiting_time);
    printf("\nAverage Turnaround Time = %.2f\n", average_turnaround_time);

    return 0;
}
```

Output:

